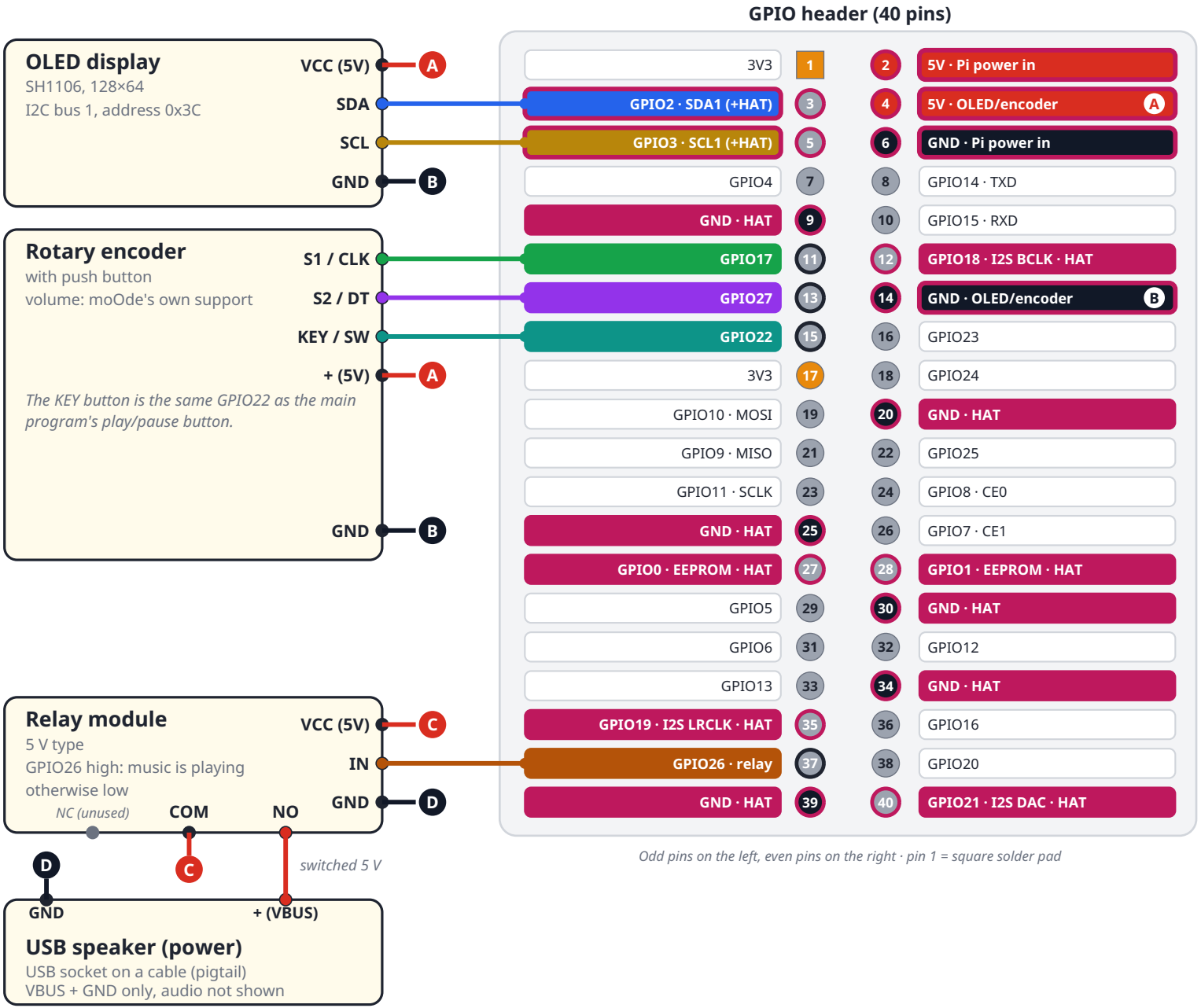
Raspberry Pi Zero 2 W – wiring

PLAN ONLY – not yet verified on the hardware

Waveshare PCM5122 Audio HAT on the header + OLED (I2C) + rotary encoder (GPIO) + relay switching the USB speaker's 5 V. Pin numbers are physical (BOARD) numbers.



Legend

- I2C SDA (GPIO2)
- I2C SCL (GPIO3)
- Encoder S1/CLK (GPIO17)
- Encoder S2/DT (GPIO27)
- Button KEY (GPIO22)
- Relay control (GPIO26)
- 5V (power)
- GND
- Pin used by the Waveshare PCM5122 HAT
- A — 5V branch from pin 4: OLED, encoder
- B — GND branch from pin 14: OLED, encoder
- C — relay VCC and COM: 5V from the supply
- D — relay and speaker GND: from the supply
- magenta outline: the HAT uses it too

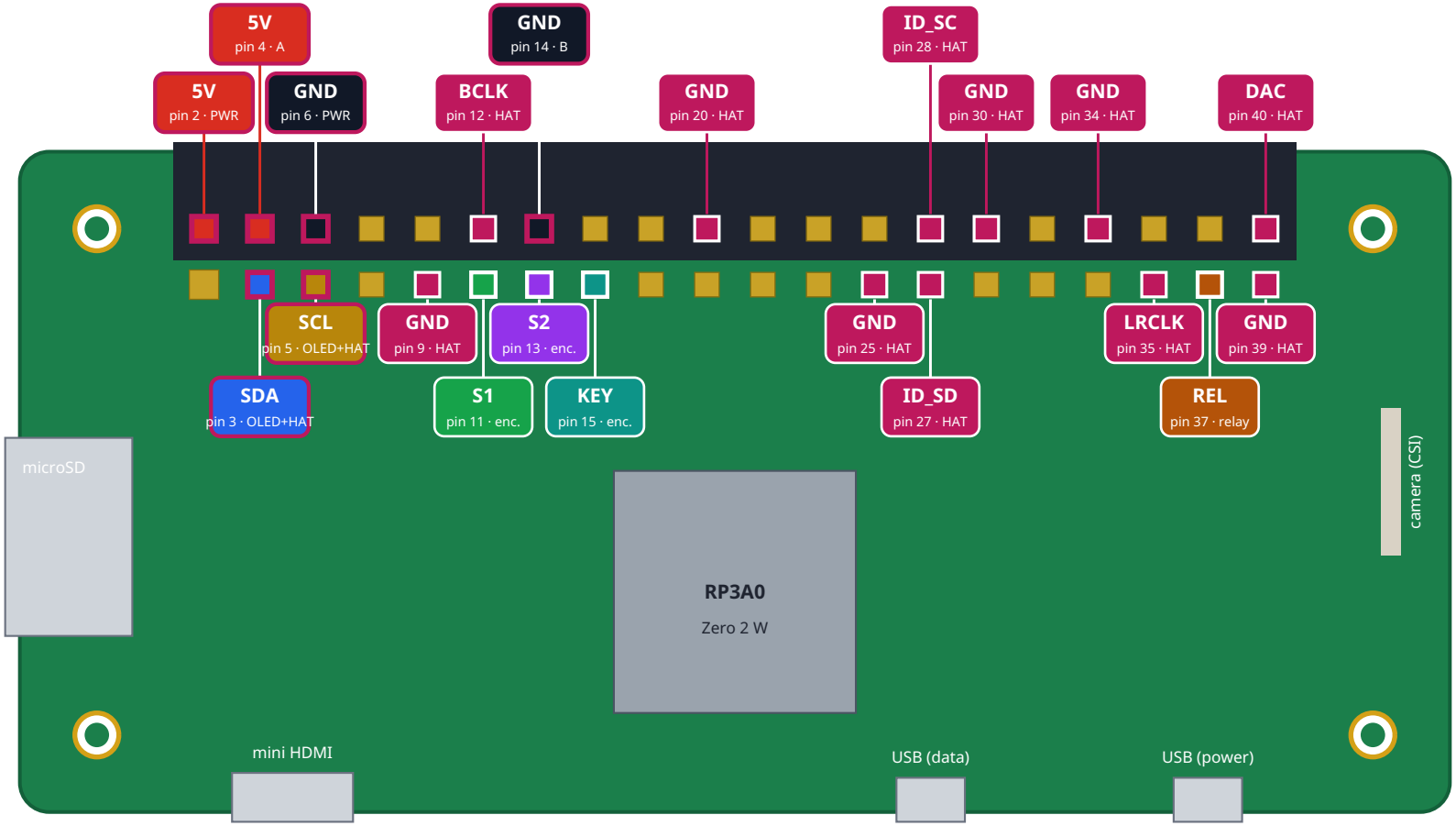
Notes

- The Pi is powered through pin 2 (5V) and pin 6 (GND); the HAT gets its power from there too.
- OLED and encoder take 5V / GND from pins 4 / 14.
- The relay switches only the speaker's +5V: supply +5V to COM, NO to the speaker's VBUS. The speaker's GND is not switched. NC is unused.
- The OLED shares the HAT's I2C bus (0x3C and 0x4C); no clash.
- GPIO17, 27, 22 (encoder) and GPIO26 (relay) are free; the HAT does not use them.
- With 5V powering, check that module pull-ups do not put 5V on the GPIOs (page 4).

Wiring on the board (top view)

Raspberry Pi Zero 2 W with the 40-pin header along the top edge. To scale (65 × 30 mm); the HAT sits on top of it.

PLAN ONLY – not yet verified on the hardware



Pin used by the Waveshare PCM5122 Audio HAT (5V, GND, I2C, I2S, EEPROM) free pin

PWR = Pi power (pin 2 5V, pin 6 GND) · A = 5V branch (pin 4) and B = GND branch (pin 14) for the OLED and the encoder · magenta outline = the HAT uses it too.
Pin 1 (square) is on the left; even pins in the top row, odd pins in the bottom row. The 3.3V pins (1, 17) stay free.

All pins and connections

Hand-wired devices and the Waveshare PCM5122 HAT. The coloured bar is the colour used in the drawings.

PLAN ONLY – not yet verified on the hardware

Device	Signal	Pi pin	BCM	Function	Source
Pi power	**5V in**	2	**5V**	powers the Pi (and the HAT) through the header	user-supplied
Pi power	**GND in**	6	**GND**	ground of the Pi's power	user-supplied
OLED	**VCC**	4	**5V**	OLED supply, 5 V (branch A)	user-supplied
OLED	**SDA**	3	**GPIO2**	I2C-1 data, address 0x3C (shared with the HAT)	good_moode/config.py (I2C_ADDR)
OLED	**SCL**	5	**GPIO3**	I2C-1 clock (shared with the HAT)	good_moode/state_machine.py
OLED	**GND**	14	**GND**	OLED ground (branch B)	user-supplied
Rotary encoder	**S1 / CLK**	11	**GPIO17**	rotation: volume, handled by moOde	moOde setting (user-supplied)
Rotary encoder	**S2 / DT**	13	**GPIO27**	second phase of the rotation, handled by moOde	moOde setting (user-supplied)
Rotary encoder	**KEY / SW (button)**	15	**GPIO22**	play/pause: mpc toggle, handled by good_moode	good_moode/config.py (BUTTON_PIN)
Rotary encoder	**+ (if present)**	4	**5V**	module supply, 5 V (branch A)	user-supplied
Rotary encoder	**GND**	14	**GND**	module ground (branch B)	user-supplied
Relay	**IN**	37	**GPIO26**	control: HIGH while playing, else LOW (2 s delay)	good_moode/hardware.py (Relay)
Relay	**VCC**	–	**5V**	5 V straight from the supply (branch C)	user-supplied
Relay	**GND**	–	**GND**	GND straight from the supply (branch D)	user-supplied
Relay	**COM**	–	**5V**	contact common: +5V from the supply (branch C)	user-supplied
Relay	**NO**	–	**5V (switched)**	normally open: to the USB speaker's VBUS	user-supplied
USB speaker	**VBUS (+)**	–	**5V (switched)**	power from the relay's NO contact	user-supplied
USB speaker	**GND**	–	**GND**	GND straight from the supply (branch D)	user-supplied
PCM5122 HAT	**5V**	2, 4	**5V**	power input (same rail as above)	Waveshare wiki
PCM5122 HAT	**GND**	6, 9, 14, 20, 25, 30, 34, 39	**GND**	ground	Waveshare wiki
PCM5122 HAT	**SDA / SCL**	3, 5	**GPIO2 / 3**	DAC control, I2C-1, address 0x4C	Waveshare wiki
PCM5122 HAT	**CLK**	12	**GPIO18**	I2S bit clock (BCLK)	Waveshare wiki
PCM5122 HAT	**LRCLK**	35	**GPIO19**	I2S word clock	Waveshare wiki
PCM5122 HAT	**DAC**	40	**GPIO21**	I2S data (Pi output)	Waveshare wiki
PCM5122 HAT	**ID_SD / ID_SC**	27, 28	**GPIO0 / 1**	EEPROM (I2C-0, 0x50): automatic driver loading	Waveshare wiki (text)

I2C buses

I2C-1 (GPIO2/3, pins 3 and 5): 0x3C = OLED · 0x4C = PCM5122 DAC (i2cdetect shows "UU" once the driver has claimed it)
I2C-0 (GPIO0/1, pins 27 and 28): 0x50 = the HAT's EEPROM; the Pi uses it only to recognise the HAT automatically

Notes and checks

Clashes, free pins, what the code does not tell, and how to check the wiring.

PLAN ONLY – not yet verified on the hardware

Power and clashes

- The Pi is powered through header pin 2 (5V) and pin 6 (GND). The HAT, and the OLED and encoder (from pins 4 and 14), run from the same rail: pins 2 and 4 are the same 5V rail, and pins 6 and 14 the same GND.
- The OLED (0x3C) and the PCM5122 DAC control (0x4C) share I2C-1 at different addresses, so they do not disturb each other.
- The HAT does not use the encoder pins (GPIO17, 27, 22) or the relay pin (GPIO26). The 3.3V pins (1, 17) stay free. Still free: GPIO4, 14, 15, 23, 24, 5, 6, 12, 13, 16, 20, among others.

The USB speaker and the relay

- The relay switches only the speaker's +5V: the supply's +5V goes to COM, NO goes to the speaker's VBUS, and the speaker's GND comes straight from the supply. NC is unused. With GPIO26 high the contact closes and the speaker is powered; with the Pi off (or GPIO26 low) the speaker is off.
- Check the module's contact rating against the speaker's current, and that the supply can carry the Pi, HAT, OLED, the relay coil and the speaker together.
- The speaker is wired through a USB socket on a cable (pigtail), with VBUS and GND only. The audio path from the DAC to the speaker is not part of this drawing.

Check: 5 V and the GPIOs (the Pi's pins tolerate 3.3 V only)

- OLED: if the module's I2C pull-ups go to its VCC (5 V), 5 V reaches GPIO2/3 at rest. Measure SDA at rest: it must not exceed 3.3 V. Many modules pull up to the 3.3 V side, but it depends on the module.
- Encoder: a bare encoder needs no "+" pin (the Pi's inputs can use their internal pull-ups; check what moOde's encoder setting does). If the module (e.g. KY-040) has pull-ups to "+" and "+" gets 5 V, the GPIO17/27/22 idle level can be 5 V. Then connect "+" to 3.3V (pin 1 or 17) or leave it unconnected; the button needs only GND.
- Relay: the module input is driven by the Pi's 3.3 V signal. Most modules switch on that, but verify. Many cheap modules are active-low (LOW closes): there HIGH would release the relay. The stated behaviour (GPIO26 high while playing) means an active-high module or inverted logic. The Pi's GND and the relay supply's GND must be common (the same supply does that).
- Powering the Pi from the header bypasses its input protection (fuse): do not plug in micro-USB power at the same time, and use a stable 5 V supply (the HAT, OLED, relay and speaker all load it).

What the code states

- OLED: luma.oled sh1106, i2c(port=1, address=0x3C), 128×64 (good_moode/config.py: I2C_ADDR, state_machine.py). Button: BUTTON_PIN = 22 (BCM), /dev/gpiochip0, internal pull-up, a press pulls the pin to GND; a press runs mpc toggle.
- Rotary encoder: the volume is controlled by moOde's own rotary encoder support (S1 = GPIO17, S2 = GPIO27 are the pins set there; in moOde: Configure > Peripherals > Rotary encoder = On, parameters 50 2 10 17 27). good_moode only shows the volume: it follows MPD's mixer events, so the bar changes as soon as moOde changes the volume.
- Relay (good_moode/config.py: RELAY_PIN = 26, good_moode/hardware.py: Relay): GPIO26 high while playing, low otherwise; it drops after 2 s of continuous idle (RELAY_OFF_DELAY) so a short pause or an mpc error does not click it, and it is switched off on shutdown.

What the code does not state

- Whether the HAT has a pass-through header: the Waveshare page does not say. If not, wires go to the Pi Zero's solder pads on the back or to a stacking header.
- The sound card is called IQaudIODAC according to Waveshare; pick the matching I2S device in moOde. Checks: aplay -l, i2cdetect -y 1 (3C and 4C or UU), pinctrl get 17,22,26,27.
- GPIO22 (the encoder's KEY) is the play/pause button of good_moode. Do not assign the same pin to the encoder's switch in moOde's settings, otherwise both react to one press (a double toggle). experiment/rotary_encoder_test.py is obsolete (moOde handles the encoder); do not run it next to moOde's encoder setting.